

Hướng dẫn Cài đặt từ A → Z

Tài liệu này hướng dẫn cài đặt hệ thống Dental AI Assistant từ máy trắng, cho cả Windows, macOS và Linux. Mỗi bước đều có phần **kiểm chứng** (verify) để đảm bảo đúng trước khi chuyển bước.

Ước tính thời gian cài đặt: **~45 phút** nếu mạng ổn định (download model sbert ~420 MB + Ollama model ~1 GB + `node_modules`).

1. Yêu cầu hệ thống tối thiểu

Thành phần	Khuyến nghị	Tối thiểu
RAM	8 GB	4 GB (nếu chỉ dùng cloud)
VRAM (nếu dùng Ollama)	4 GB	2 GB với model Q4
Ổ cứng	10 GB trống	5 GB
OS	Windows 10+, Ubuntu 22.04+, macOS 12+	—
Kết nối mạng	Có (cho lần cài đầu)	—

2. Cài đặt công cụ nền tảng

2.1. Python 3.10+

- **Windows:** tải từ <https://python.org/downloads>, nhớ tick “Add Python to PATH”.
- **macOS:** `brew install python@3.11`.
- **Ubuntu:** `sudo apt install python3.11 python3.11-venv python3-pip`.

Kiểm chứng:

```
python --version      # >= 3.10.x
pip --version
```

2.2. Node.js 18+ và npm

- Tải LTS từ <https://nodejs.org> (đã bao gồm npm).
- Hoặc dùng `nvm` / `fnm` để quản lý phiên bản.

Kiểm chứng:

```
node --version        # >= v18
npm --version
```

2.3. Git

Tải từ <https://git-scm.com>. Trên Windows nhớ tick "Git Bash Here".

Kiểm chứng: `git --version`.

2.4. MySQL 8.0+ hoặc MariaDB 10.6+

Chọn 1 trong 3 cách:

Cách	Khi nào dùng
Cài MySQL Server chính thức (https://dev.mysql.com/downloads/installer/)	Windows, có root quyền
XAMPP (https://www.apachefriends.org)	Sinh viên quen dùng phpMyAdmin
Docker (<code>docker run -p 3306:3306 -e MYSQL_ALLOW_EMPTY_PASSWORD=true mysql:8</code>)	Không muốn cài native

Kiểm chứng:

```
mysql -u root -p -e "SELECT VERSION();"
```

2.5. (Tùy chọn) Ollama — nếu muốn chạy LLM local

- Tải: <https://ollama.com/download>.
- **Khuyến nghị GPU NVIDIA** để tốc độ chấp nhận được (≥ 4 GB VRAM).
- CPU-only vẫn chạy được nhưng phản hồi chậm (~15–30 s/câu).

Kiểm chứng: `ollama --version`.

3. Clone mã nguồn & Tạo môi trường Python

```
git clone https://github.com/naminc/DA14_AI_Dental_Agent.git
cd DA14_AI_Dental_Agent

# Tạo môi trường ảo
python -m venv .venv

# Kích hoạt
# Windows PowerShell:
.venv\Scripts\Activate.ps1
# Windows CMD:
.venv\Scripts\activate.bat
# macOS/Linux:
source .venv/bin/activate
```

```
# Cài thư viện
pip install --upgrade pip
pip install -r requirements.txt
```

Lỗi hay gặp trên Windows: `Activate.ps1` bị chặn bởi Execution Policy. Fix:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Kiểm chứng:

```
python -c "import fastapi, faiss, openai, sentence_transformers, underthesea;
print('OK')"
```

4. Tạo & Cấu hình file `.env`

```
cp .env.example .env      # macOS/Linux
copy .env.example .env    # Windows
```

4.1. Sinh `SECRET_KEY` an toàn

```
python -c "import secrets; print(secrets.token_hex(32))"
```

→ copy chuỗi 64 ký tự hex vào biến `SECRET_KEY` trong `.env`.

4.2. Điền các biến còn lại

Mẫu tối thiểu để chạy **cloud-only** (OpenAI cho cả embedding + LLM):

```
EMBEDDING_ENGINE=openai
LLM_ENGINE=openai
OPENAI_API_KEY=sk-proj-XXXXXXXXXXXXXXXXXXXXX
OPENAI_CHAT_MODEL=gpt-4.1-mini
TOP_K=10
DATABASE_URL=mysql+pymysql://root:yourpassword@localhost:3306/dental_agent_db
SECRET_KEY=<chuỗi sinh ở bước 4.1>
ACCESS_TOKEN_EXPIRE_MINUTES=1440
ALGORITHM=HS256
```

Mẫu tối thiểu để chạy **local-only** (miễn phí, không cần OpenAI API key):

```
EMBEDDING_ENGINE=local
LLM_ENGINE=local
OLLAMA_BASE_URL=http://localhost:11434/v1
OLLAMA_CHAT_MODEL=qwen2.5:1.5b-instruct-q4_K_M
TOP_K=10
DATABASE_URL=mysql+pymysql://root:yourpassword@localhost:3306/dental_agent_db
SECRET_KEY=<chuỗi sinh ở bước 4.1>
ACCESS_TOKEN_EXPIRE_MINUTES=1440
ALGORITHM=HS256
```

Cấu hình **hybrid** (local embedding + cloud LLM) được khuyến nghị cho demo bảo vệ: tiết kiệm chi phí embedding, giữ chất lượng reasoning cao.

5. Tạo Database MySQL

5.1. Tạo database trống (tên phải khớp với `DATABASE_URL`)

```
CREATE DATABASE dental_agent_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

Hoặc qua CLI:

```
mysql -u root -p -e "CREATE DATABASE dental_agent_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;"
```

5.2. (Tùy chọn) Tạo user riêng cho ứng dụng

```
CREATE USER 'dental_app'@'localhost' IDENTIFIED BY 'your-strong-password';
GRANT ALL PRIVILEGES ON dental_agent_db.* TO 'dental_app'@'localhost';
FLUSH PRIVILEGES;
```

Rồi đổi `DATABASE_URL=mysql+pymysql://dental_app:your-strong-password@localhost:3306/dental_agent_db`.

5.3. Kiểm chứng kết nối

```
python -c "from sqlalchemy import create_engine; import os; from dotenv import
load_dotenv; load_dotenv(); create_engine(os.getenv('DATABASE_URL')).connect();
print('DB OK')"
```

Các bảng `users`, `chat_sessions`, `messages` sẽ được **tự tạo** khi backend start lần đầu (`models.Base.metadata.create_all` trong `api/main.py`). Không cần chạy migration thủ công.

6. Xây dựng FAISS Index (bắt buộc, chạy 1 lần cho mỗi engine)

6.1. Nếu dùng `EMBEDDING_ENGINE=local`

```
python -m src.retriever.ingest --engine local
```

Lần đầu sẽ tự tải model `keepitreal/vietnamese-sbert` (~420 MB) về cache HuggingFace (`~/.cache/huggingface/`). Sau đó mã hóa 762 tài liệu, output vào `data/vector_db/local/`:

```
data/vector_db/local/
├── faiss.index      # FAISS IndexFlatIP, 768 dim
└── metadata.json   # 762 docs
```

6.2. Nếu dùng `EMBEDDING_ENGINE=openai`

```
python -m src.retriever.ingest --engine openai
```

- Cần `OPENAI_API_KEY` hợp lệ.
- Chi phí: ~0.02 USD/1M tokens. Toàn bộ 762 bài ~1M tokens → **~0.02 USD**.
- Output tại `data/vector_db/openai/` (1536 dim).

6.3. Có thể build cả hai cùng lúc

Index được lưu tách biệt (`vector_db/local/` vs `vector_db/openai/`). Thay đổi `.env EMBEDDING_ENGINE` là tự động load đúng index, **không cần ingest lại**.

Kiểm chứng:

```
python -c "import faiss; idx =
faiss.read_index('data/vector_db/local/faiss.index'); print(f'Vectors:
{idx.ntotal}, Dim: {idx.d}')"

```

Kỳ vọng: `Vectors: 762, Dim: 768` (local) hoặc `Vectors: 762, Dim: 1536` (openai).

7. (Chỉ khi dùng LLM local) Cài Ollama & Pull model

7.1. Pull model đúng theo `.env`

```
ollama pull qwen2.5:1.5b-instruct-q4_K_M
```

Kích thước ~1 GB. Có thể chọn model khác bằng cách:

- 1. Pull model mong muốn: `ollama pull qwen2.5:7b-instruct-q4_K_M` (cần 6 GB VRAM).
- 2. Sửa `OLLAMA_CHAT_MODEL` trong `.env`.

7.2. Bật Ollama với cấu hình tối ưu

Windows — chạy `start_ollama.bat` ở root dự án. Script set các biến sau trước khi gọi `ollama serve`:

Biến	Giá trị	Mục đích
<code>OLLAMA_NUM_GPU</code>	999	Đẩy hết layers lên GPU nếu đủ VRAM
<code>OLLAMA_FLASH_ATTENTION</code>	1	Tăng tốc attention computation
<code>OLLAMA_KEEP_ALIVE</code>	60m	Giữ model trong RAM/VRAM 60 phút, tránh cold-start
<code>OLLAMA_NUM_PARALLEL</code>	3	Cho phép 3 request chạy song song (vì pipeline có 2 LLM call song song: Extract + Expand)
<code>OLLAMA_MAX_LOADED_MODELS</code>	1	Chỉ load 1 model, tiết kiệm VRAM

macOS/Linux — tương đương:

```
OLLAMA_NUM_GPU=999 \  
OLLAMA_FLASH_ATTENTION=1 \  
OLLAMA_KEEP_ALIVE=60m \  
OLLAMA_NUM_PARALLEL=3 \  
OLLAMA_MAX_LOADED_MODELS=1 \  
ollama serve
```

Kiểm chứng:

```
curl http://localhost:11434/api/tags  
# Hoặc trên Windows PowerShell:  
Invoke-RestMethod http://localhost:11434/api/tags
```

→ phải trả về JSON chứa tên model bạn vừa pull.

8. Smoke-test Backend

```
uvicorn api.main:app --reload --port 8000
```

Log mong đợi:

```
[STARTUP] Đang khởi tạo DentalChatbot + load Embedding model...
[STARTUP] DentalChatbot khởi tạo xong trong X.XXs
[STARTUP] Sẵn sàng nhận request.
INFO:      Uvicorn running on http://127.0.0.1:8000
```

Truy cập <http://127.0.0.1:8000/docs> — giao diện Swagger UI hiển thị toàn bộ endpoints.

Test endpoint mở (register):

```
curl -X POST http://127.0.0.1:8000/api/auth/register \
-H "Content-Type: application/json" \
-d '{"full_name":"Test User","email":"test@demo.com","password":"123456","confirm_password":"123456"}'
```

Kỳ vọng HTTP 201 với JSON `{"id":1,"full_name":"...", "email":"...", "is_2fa_enabled":false}`.

9. Cài đặt Frontend Next.js

```
cd frontend/nextjs-app

# npm (mặc định)
npm install
npm run dev

# hoặc pnpm (nhanh hơn, đã có pnpm-lock.yaml)
pnpm install
pnpm dev
```

Kiểm chứng: mở <http://localhost:3000>, thấy màn hình login/đăng ký của Dental AI.

Biến môi trường frontend (sẵn trong `frontend/nextjs-app/.env.local`):

```
NEXT_PUBLIC_API_URL=http://127.0.0.1:8000/api
```

Nếu backend chạy trên máy khác, cập nhật URL này rồi **restart** `npm run dev` (Next.js chỉ đọc `.env.local` khi start).

10. (Tùy chọn) Nâng cấp dataset v1 → v2

Nếu muốn tự sinh lại `data/raw/dental_dataset_v2.json`:

```
# Cần OPENAI_API_KEY + UPGRADE_DATASET_MODEL trong .env
python tools/upgrade_dataset.py
```

- Script dùng `gpt-4o-mini` (rẻ, ổn định) để sinh `summary` + `content_md`.
- Tự lưu checkpoint mỗi 50 bài → có thể dừng và chạy tiếp (`resume-safe`).
- Chi phí toàn bộ 762 bài: **≈ 0.5–1 USD** tùy độ dài content.

Sau khi có `dental_dataset_v2.json`, chạy lại ingest để FAISS index dùng dữ liệu mới:

```
python -m src.retriever.ingest --engine local
```

11. Các lỗi thường gặp & cách xử lý

Lỗi	Nguyên nhân	Cách xử lý
<code>ValueError: Thieu DATABASE_URL trong file .env</code>	Thiếu hoặc sai tên biến	Kiểm tra <code>.env</code> có <code>DATABASE_URL=mysql+pymysql://...,</code> nhớ <code>load_dotenv()</code> đã chạy
<code>RuntimeError: SECRET_KEY chưa được cấu hình</code>	Biến rỗng	Sinh key bằng <code>secrets.token_hex(32)</code> → cập nhật <code>.env</code>
<code>FileNotFoundError: FAISS index not found</code>	Chưa chạy ingest	<code>python -m src.retriever.ingest --engine <local openai></code>
<code>ConnectionError: KHÔNG THỂ KẾT NỐI TỚI OLLAMA</code>	Ollama chưa bật hoặc sai URL	Chạy <code>start_ollama.bat</code> ; kiểm tra <code>OLLAMA_BASE_URL</code> phải đúng <code>http://localhost:11434/v1</code>
<code>ValueError: Thiếu OPENAI_API_KEY trong .env</code>	<code>LLM_ENGINE=openai</code> nhưng chưa có key	Điền key hoặc đổi <code>LLM_ENGINE=local</code>
Frontend báo <code>Network Error / CORS</code>	Backend chưa chạy hoặc cổng sai	Đảm bảo backend ở <code>http://127.0.0.1:8000</code> + <code>NEXT_PUBLIC_API_URL</code> khớp
<code>Access denied for user 'root'@'localhost'</code>	Sai password MySQL	Sửa <code>DATABASE_URL</code> đúng <code>username/password</code>
<code>ImportError: Could not find the DLL(s) 'libfaiss_avx2.dll'</code>	Python/FAISS không tương thích	Cài lại <code>pip install faiss-cpu --no-cache-dir</code> , dùng Python 3.10/3.11
<code>from underthesea import word_tokenize</code> chạy chậm lần đầu	Underthesea lazy-load model	Bình thường — chỉ chậm lần đầu ingest, các lần sau đã cache

12. Tổng kết checklist cài đặt

- ☐ Python 3.10+ + `.venv` activated + `pip install -r requirements.txt`
- ☐ Node 18+ + `npm install` trong `frontend/nextjs-app`
- ☐ MySQL đang chạy + database `dental_agent_db` đã được tạo
- ☐ `.env` có đủ: `DATABASE_URL`, `SECRET_KEY`, `TOP_K`, engine keys
- ☐ FAISS index đã build cho engine chọn (`data/vector_db/<engine>/faiss.index` tồn tại)
- ☐ (Nếu local LLM) Ollama đã bật + model đã pull
- ☐ Backend chạy được tại `http://127.0.0.1:8000/docs`
- ☐ Frontend chạy được tại `http://localhost:3000`

Sau khi checklist xanh hoàn toàn, chuyển sang `RUN_GUIDE.md` để biết cách bật/tắt hệ thống gọn gàng mỗi lần làm việc.